



BackTracking

Öğrenci Adı: Gökhan Dumlupınar

Öğrenci Numarası: 23011503

Dersin Öğretmeni: M. Elif Karşılıgil

Video Linki: <https://youtu.be/-nbKaoPJfml>

1. Problemin Çözülmesi

Kabinler ve serverlara özel structlar oluşturuldu. Kabinlerimize serverların yerleştirilmesi belli kuralları sağlaması gerekiyor: Çakışmalar, kabinin toplam alabilceği kapasite, ve en oluşan ısı/güç oranı kontrol edilmeli. Kapışma kontrolü için özel bir matris açtım böylece direct olarak oluşan matrsten direkt çakışma var mı kontrolü yapabildim.

Backtrack fonksiyonu:

Algoritma state-space ağacını derinlik öncelikli arama (DFS) stratejisiyle keşfetmekte ve ilk

```
--- Conflict info ---
Conflict num: 1
Conflict 1 (S1 S2): 1 2

---Conflict Matrix ---
S1S2S3S4
S1 0  1  0  0
S2 1  0  0  0
S3 0  0  0  0
S4 0  0  0  0
```

geçerli çözümü bulduğunda found değişkenini 1 yaparak aramayı sonlandırmaktadır. Sırayla serverları kabine – Kapasite ve çakışma var mı kontrolünü yaptırıyorum burada geçerli olmasa o dal atlanır böylece bu kurallara uygun şekilde yerleştiriyor . Oluşan sonucun oranını en son control s4 kontrolünün ardından tekrar fonksiyonu çağırır ama bu kez sadece kabinlerin oranını control eder. Kontrolden geçemezse removeServer ile aslında önceki dalın seneryosuna giriş yapar.

2. Karşılaşılan Sorunlar

Struct mantığını hızlı oluşturdum ancak backtrackingi nasıl yapacağımı oluşturmak zor oldu. İlk denemelerde tüm kontrollerin sunucu yerleştirilmeden önce yapılması planlanmıştır. Kapasite, çakışma ve ısı oranı kontrollerinin hepsinin addServer fonksiyonundan önce gerçekleştirilmesi düşünülmüştür. Ancak bu yaklaşımda ciddi bir sorun ortaya çıkmıştır. Isı oranı kontrolü ara adımlarda yapıldığında tek başına "sıcak" olan sunucular hiçbir kabine yerleştirilememektedir. Örneğin ratio değeri 1.50 olan bir sunucu tek başına limit üstünde olduğu için reddedilmekte, ancak bu sunucunun başka bir "soğuk" sunucu ile birlikte yerleştirildiğinde geçerli bir kombinasyon oluşturabileceği görülememektedir. Bu durum algoritmanın geçerli çözümleri atlamasına neden olmaktadır. Zamanla kontrol mekanizmasının iki aşamalı olması gerektiği anlaşılmıştır. Kapasite ve çakışma kontrollerin kontrolü için yerleştirme öncesinde yapılırken, ısı oranı kontrolünün tüm sunucular yerleştirildikten sonra yapılmasının daha mantıklı olduğu fark edilmiştir. Bu değişiklik algoritmanın doğruluğunu önemli ölçüde artırmıştır.

Bir diğer önemli sorun ise algoritmanın ilk geçerli çözümü bulduktan sonra durmaması olmuştur. Başlangıç versiyonunda found değişkeni kullanılmamaktaydı ve algoritma tüm state-space ağacını taramaya devam etmekteydi. Bu durumda çıktıda geçerli çözümlerin yanı sıra hiçbir sunucunun yerleştirilmediği veya sadece bazı sunucuların yerleştirildiği geçersiz durumlar da yazdırılmaktaydı.

3. Karmaşıklık Analizi

Function backtrack(serverIdx,servers[], cabins[], conflictMatrix[], serverNum, cabinNum, maxCapacity, kValue, found):

```

If serverIdx == serverNum THEN
    Valid == True
    FOR j = 0 TO cabinNum -1 DO
        IF cabins[j].serversNum > 0 THEN
            Ratio= cabins[j].totalHeat / cabins[j].totalPower
            IF ratio > kValue THEN
                valid = False
            END IF
        END IF
    END IF
END FOR
IF valid THEN
    PRINT solutionn
    found = ture
END IF
RETURN
END IF

FOR I = 0 TO cabinNUM -1 DO
    IF checkCapacity[cabins[i], servers[serverIdx] AND checlConflict(serverIdx,
cabins[i]) THEN
        addServer(serverIdx +1, found)
        backtrack(serverIdx + 1, found)
        removeServer(cabins[i], severs[serverIdx])
    END IF
END FOR

// backtrack(...)

```

END FUNCTION

S1 -> k1 ,k2 , k3 , s2-> k1,k2,k3 ...

Ağaç yapisi son derinlik-> KabinSayisi ^ServerSayisi

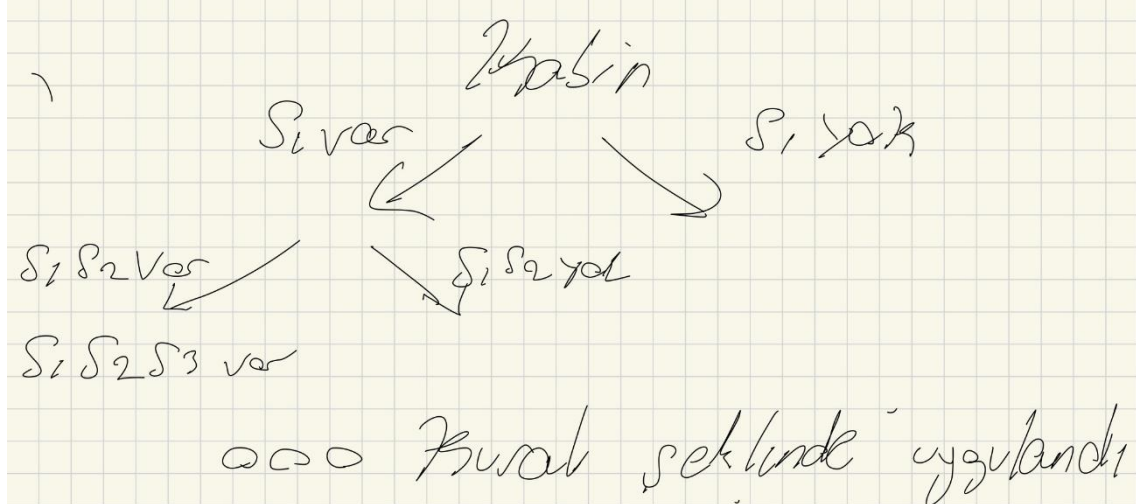
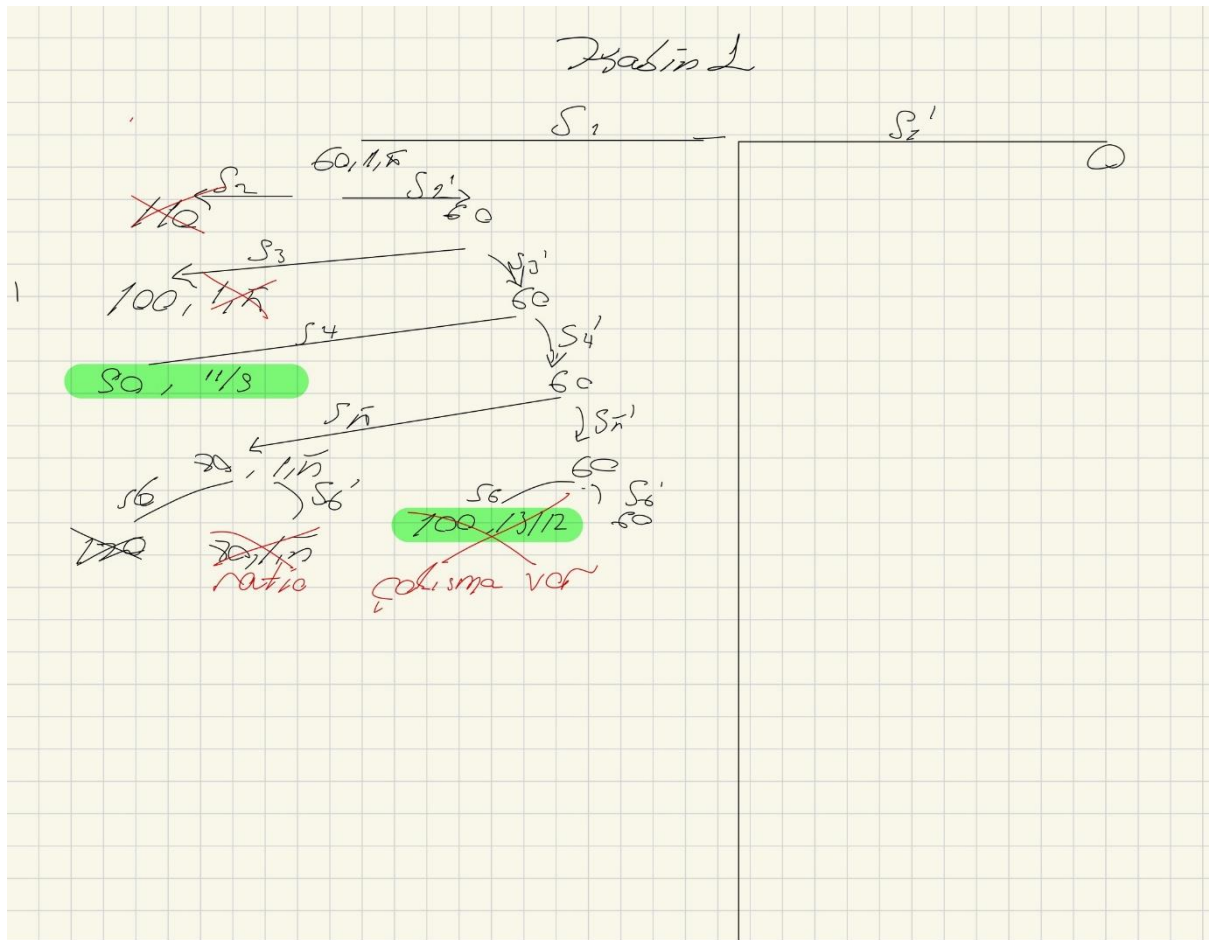
$$\rightarrow \sum_{N=1}^{ServerSayisi} KabinSayisi^N$$

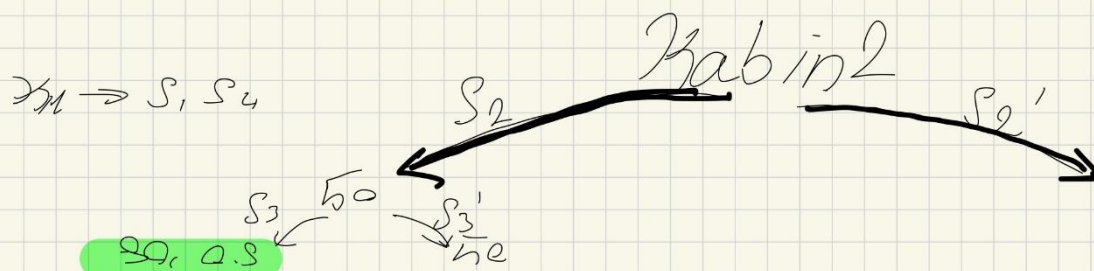
➔ Yaklaşık $O((\text{KabinSayisi})^{(\text{ServerSayisi}+1)})$

////////////////////////////////////

2. kısım aşağıdadır:

2. Kısım





$\lambda_2 \rightarrow S_2 S_3 \quad 10/8 \quad \checkmark$
 $\lambda_3 \rightarrow S_5 S_6 \quad 7/6 \quad \checkmark$

2. Soru a) Toplam güç $60 + 50 + 40 + 30 + 20 + 40 = 240W$
Kabin Kapasitesi = $100W$

$$LB = \frac{\text{Toplam güç}}{\text{Kabin Kapasitesi}} \\ \frac{240}{100} \\ \frac{2,4}{1} \\ = 2,4 \text{ Kabin}$$

b) Kabin sürücüler S_1, S_2, S_3 $80W$ 'dır,
 $80/100 = 0,8$
Kabin

Senaryo A: $S_1, S_2 \rightarrow$ Kabin 1 $110W$ X
 $S_1, S_3 \rightarrow$ Kabin 1 $100W$ $1,5$ ratio X

$S_1 \rightarrow$ Kabin 1

$S_2, S_3 \rightarrow$ Kabin 2 $80W$ $1,1$ ratio ✓

S_6 $\rightarrow$ Kabin 1'e gidemez 2'ye'de gidemez

Soru 3)

	w	H	H/w
S ₄	30	20	0.6
S ₂	50	40	0.8
S ₆	40	40	1
S ₁	60	80	1.3
S ₃	40	60	1.5
S ₅	20	30	1.5

tek başlarına değerler

α minimum değer sayısının 3 olduğuna gelmiştik

2₁ $\frac{S_1 S_4}{80W \cdot 1.2} \checkmark$ $\frac{S_1 S_2}{110W \times}$ S_1 'i denendi yetli diğeri sadece 2'nin 3'ü

2₂ a) $\frac{S_3 S_2}{80W \cdot 1.1} \checkmark$ b) $\frac{S_3 S_6}{80W \cdot 1.2} \checkmark$
c) $\frac{S_3 S_6 S_5}{100W \cdot 1.5} \checkmark$

2₃ a) $\frac{S_5 S_6}{80W \cdot 1.1} \checkmark$ b) $\frac{S_2 S_5}{70W \cdot 1} \checkmark$

c) $\frac{S_2}{50W \cdot 0.8} \checkmark$